TEE 환경에서 스마트 컨트랙트 실행 및 검증 프로세스

개요

Sapphire ParaTime은 Oasis Network의 공식 기밀 ParaTime으로, TEE(Trusted Execution Environment) 환경에서 EVM 호환 스마트 컨트랙트를 실행하고 검증하는 시스템입니다. 이 문서는 각 프로세스가 어떻게 진행되는지 상세히 설명합니다.

1. 시스템 아키텍처

1.1 핵심 컴포넌트

- **Runtime**: Rust로 구현된 TEE 실행 환경
- **Contracts**: Solidity로 구현된 기밀성 보장 스마트 컨트랙트
- **Clients**: 다양한 언어(Go, Python, TypeScript)로 구현된 클라이언트 라이브러리
- **Integrations**: 기존 Ethereum 도구들과의 통합 플러그인

1.2 TEE 환경 구성

```
┌─────────────────────────────────────┐
│          TEE (SGX Enclave)          │
│ ┌─────────────────────────────────┐ │
│ │         Sapphire Runtime        │ │
│ │ ┌─────────────────────────────┐ │ │
```

```
| | |         EVM Module          | | |
| | |        - 기밀성 보장         | | |
| | |        - 암호화 프리컴파일    | | |
| | |        - 상태 암호화         | | |
| | └──────────────────────────┘ | |
| └──────────────────────────────┘ |
└────────────────────────────────┘
```

2. 트랜잭션 처리 프로세스

2.1 트랜잭션 제출 단계

1. **클라이언트 요청**

- 사용자가 dApp을 통해 트랜잭션 생성
- 클라이언트 라이브러리가 트랜잭션을 암호화

2. **Calldata 암호화**

```
// CalldataEncryption.sol에서 구현
function  encryptCallData(bytes memory  in_data)
view  returns (bytes memory  out_encrypted)
```

- X25519 키 교환을 통한 대칭키 생성
- DeoxysII 알고리즘으로 calldata 암호화
- CBOR 형식으로 메타데이터 포함

2.2 TEE 내부 처리

1. **트랜잭션 수신 및 복호화**

```
// runtime/src/lib.rs
impl sdk::Runtime for Runtime {
type Modules = (
modules::core::Module<Config>,
modules::accounts::Module,
module_evm::Module<Config>,
// ... 기타 모듈들
);
}
```

2. **EVM 실행**

- 기밀성이 보장된 EVM 환경에서 스마트 컨트랙트 실행
- 상태 변경사항은 암호화되어 저장
- 가스 계산 및 실행 결과 생성

3. 기밀성 보장 메커니즘

3.1 암호화 프리컴파일

Sapphire는 다양한 암호화 기능을 프리컴파일로 제공:

```
library Sapphire {
```

```
// 랜덤 바이트 생성

address internal  constant RANDOM_BYTES = 0x0100000000000000000000000000000000

// 키 유도

address internal  constant DERIVE_KEY = 0x0100000000000000000000000000000000

// 암호화/복호화

address internal  constant ENCRYPT = 0x0100000000000000000000000000000000003;

address internal  constant DECRYPT = 0x0100000000000000000000000000000000004;

// 디지털 서명

address internal  constant GENERATE_SIGNING_KEYPAIR = 0x01000000000000000000000

address internal  constant SIGN_DIGEST = 0x0100000000000000000000000000000000

address internal  constant VERIFY_DIGEST = 0x0100000000000000000000000000000000

}
```

3.2 상태 암호화

- 모든 스마트 컨트랙트 상태는 TEE 내부에서 암호화되어 저장
- 외부에서는 암호화된 상태만 관찰 가능
- 실행 중에만 TEE 내부에서 복호화되어 처리

4. 검증 프로세스

4.1 SGX 증명 (Attestation)

1. **원격 증명**

```
#[cfg(target_env = "sgx")]
fn consensus_trust_root() -> Option<TrustRoot> {
// Mainnet/Testnet에 따른 신뢰 루트 설정
Some(TrustRoot {
height: 22327937,
hash: "01670e0af13fa4ceaeee8bd9672374d27a889b9304c44ab9acfd7f97663ee1b9".into(),
runtime_id: "0000000000000000000000000000000000000000000000000f80306c9858e7279".i
chain_context: "bb3d748def55bdfb797a2ac53ee6ee141e54cd2ab2dc2375f4a0703a178e6e55
})
}
```

2. **신뢰할 수 있는 서명자 검증**

```
fn trusted_signers() -> Option<TrustedSigners> {
let tps = keymanager::trusted_policy_signers();
Some(TrustedSigners {
signers: tps.signers.into_iter().map(|s| PublicKey(s.0)).collect(),
threshold: tps.threshold,
})
}
```

4.2 합의 검증

1. **블록 생성 및 검증**

- TEE 내부에서 실행된 트랜잭션 결과를 블록에 포함
- 다른 검증자 노드들이 블록 유효성 검증
- 합의 알고리즘을 통한 최종 확정

2. **상태 일관성 검증**

- 암호화된 상태의 해시값을 통한 일관성 확인
- 롤백 및 재실행을 통한 결정론적 검증

5. 보안 특성

5.1 기밀성 (Confidentiality)

- **실행 기밀성**: 트랜잭션 실행 과정이 TEE 외부에서 관찰 불가
- **상태 기밀성**: 스마트 컨트랙트 상태가 암호화되어 저장
- **입력 기밀성**: 트랜잭션 입력 데이터가 end-to-end 암호화

5.2 무결성 (Integrity)

- **실행 무결성**: TEE 환경에서 코드 변조 방지
- **상태 무결성**: 암호화된 상태의 무단 수정 방지
- **합의 무결성**: 분산 합의를 통한 결과 검증

5.3 가용성 (Availability)

- **분산 실행**: 여러 검증자 노드에서 동시 실행
- **장애 복구**: 노드 장애 시 자동 복구 메커니즘
- **확장성**: 병렬 처리를 통한 높은 처리량

6. 개발자 도구 및 통합

6.1 클라이언트 라이브러리

- **TypeScript**: `@oasisprotocol/sapphire-paratime`
- **Go**: `github.com/oasisprotocol/sapphire-paratime`
- **Python**: Python 클라이언트 라이브러리

6.2 개발 도구 통합

- **Hardhat**: `@oasisprotocol/sapphire-hardhat`
- **Ethers.js**: `@oasisprotocol/sapphire-ethers-v6`
- **Wagmi**: `@oasisprotocol/sapphire-wagmi-v2`
- **Viem**: `@oasisprotocol/sapphire-viem-v2`

7. 네트워크 구성

7.1 체인 ID 구성

```
const fn chain_id() -> u64 {
if option_env!("OASIS_UNSAFE_USE_LOCALNET_CHAINID").is_some() {
0x5afd // Localnet
} else if is_testnet() {
0x5aff // Testnet
} else {
0x5afe // Mainnet
```

```
  }

}
```

7.2 가스 정책

- **최소 가스 가격**: 100 Gwei (100,000,000,000)
- **동적 가스 가격**: 블록 사용률에 따른 자동 조정
- **최대 배치 가스**: 15,000,000

결론

Sapphire ParaTime은 TEE 기술을 활용하여 기밀성을 보장하면서도 EVM 호환성을 유지하는 혁신적인 스마트 컨트랙트 실행 환경입니다. 암호화 프리컴파일, 상태 암호화, SGX 증명 등의 기술을 통해 기존 Ethereum 생태계와의 호환성을 유지하면서도 강력한 프라이버시 보호 기능을 제공합니다.